FACULTAD DE ARQUITECTURA Y URBANISMO
UNIVERSIDAD DE CHILE

Cazador de colores

Dispositivo lúdico

USO PROLONGADO DE PANTALLAS EN NIÑOS DE 7 A 10 AÑOS

Electivo LOW-TECH

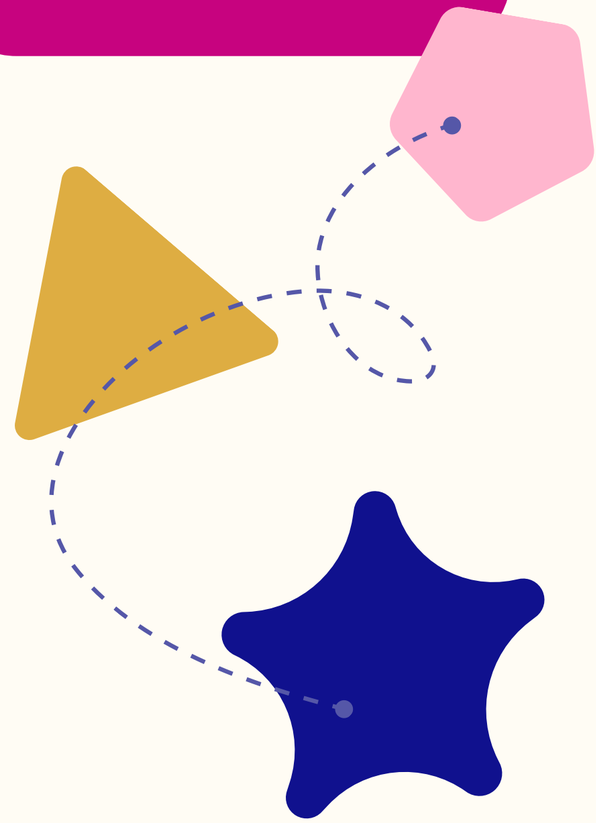
Antonia Carreño - Martina Baza - Verónica González - Xiwei Zhang

PROBLEMÁTICA

Uso prolongado de pantallas → **Dependencia a dispositivos** → **Impacto en desarrollo socioemocional**

HIPÓTESIS DEL PROYECTO

El uso de tecnologías adecuadas y adaptadas para preadolescentes de 7 a 10 años puede contribuir a disminuir los efectos negativos asociados a la dependencia y el uso excesivo de pantallas.



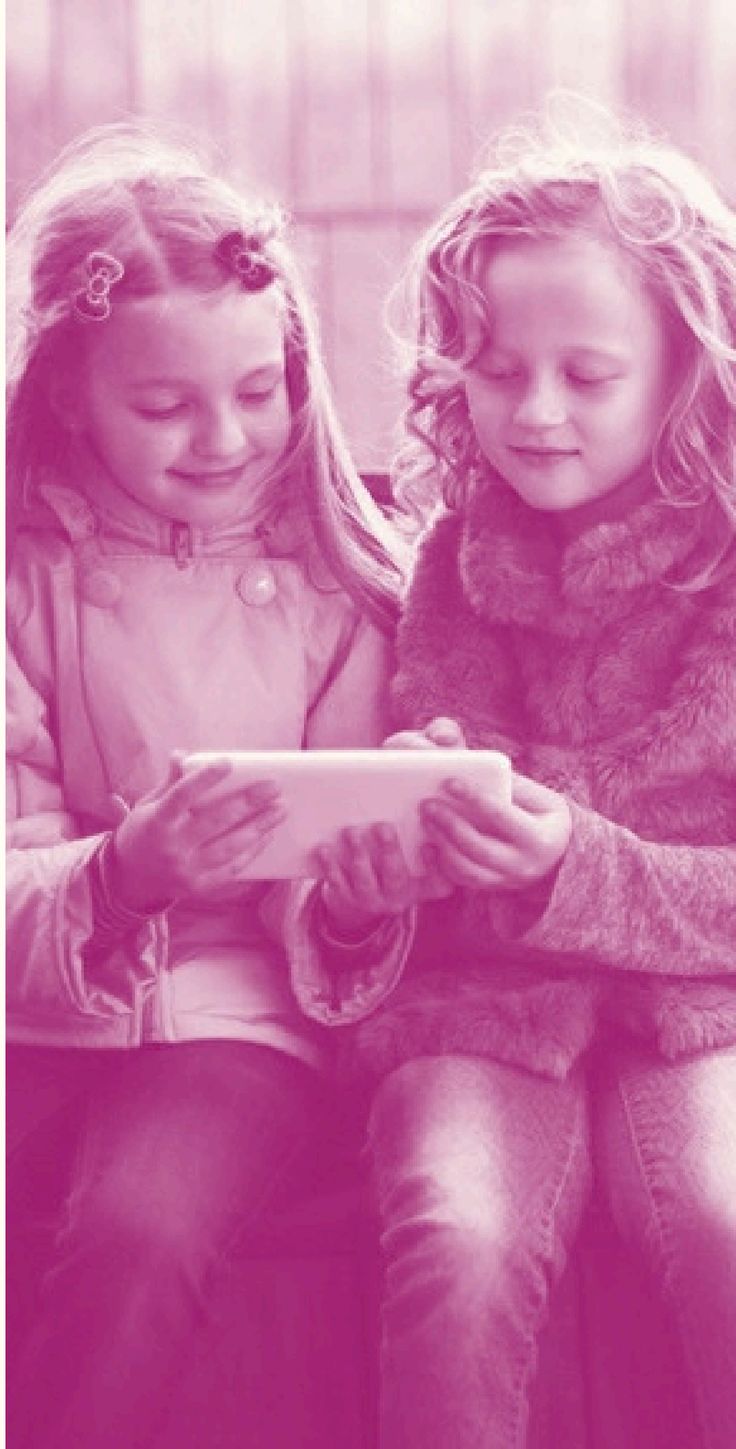
OBJETIVO DEL PROYECTO

Desarrollar un dispositivo lúdico dirigido a preadolescentes de 7 a 10 años que promueva la interacción social y exploración a través del juego colaborativo.

HIPÓTESIS DE SOLUCIÓN

Dispositivo con cámara integrada que combine los fundamentos de los juegos **"Veo veo"** y **"Búsqueda del tesoro"** pero enfocado a la recolección de elementos espaciales, iniciando en primera fase con la **búsqueda de colores** y pudiendo proyectarse a texturas, sonidos, entre otros.

Se podrá usar entre usuarios de **7 a 10 años**, quienes pueden jugar de manera **individual** y al **vincular** los dispositivos podrán asignar al otro jugador el color a buscar. El dispositivo podrá identificar el color captado y se establecerán distintos niveles.



REFERENTES

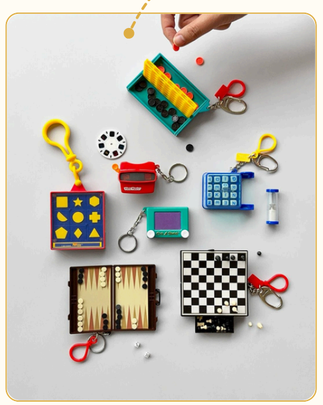
Cube World

Juguete cúbico con personajes interactivos, sensores de movimiento e imanes laterales que permitían conectar cubos e interactuar entre sí.





Juegos multijugadores portátiles no digitales





*Palabras clave

- Juegos
- Multijugador
- Conexión / vínculo
- Personaje / personificación
- Portátil / compacto
- Descubrir / explorar

Juego desplegado en múltiples pantallas pequeñas conectadas entre sí.





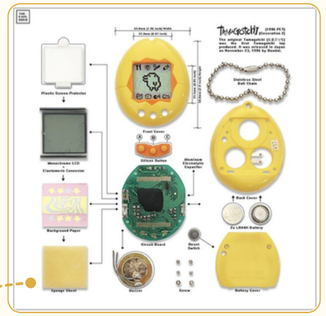
Terra

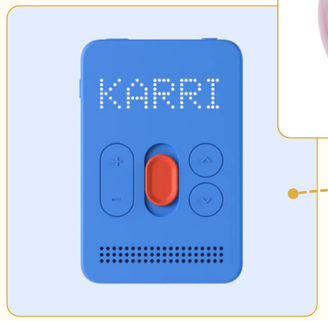
Compás digital sin pantalla que combina IA y mindfulness para guiar caminatas según el tiempo, ubicación e intención del usuario.

Pixel Chix

Dispositivo con personaje 2D que habita un mundo 3D y reacciona a las acciones del usuario. Si se descuida, puede molestarse y abandonar su casa. Algunos modelos permitían ampliar y conectar casas entre dispositivos.







Karri

Dispositivo que permite la comunicación y conexión entre niños y adolescentes sin pantallas.

Tamagotchi

Mascota digital portátil en forma de huevo que requiere cuidados desde su nacimiento hasta su desarrollo. Si se descuida, puede enfermar y morir.



Starboy

Dispositivo portátil con animaciones interactivas que responden a gestos. Puede llevarse como accesorio en bolsos, mochilas o cinturones.

DIAGRAMA INICIAL Y COMPONENTES SISTEMA

Vista frontal

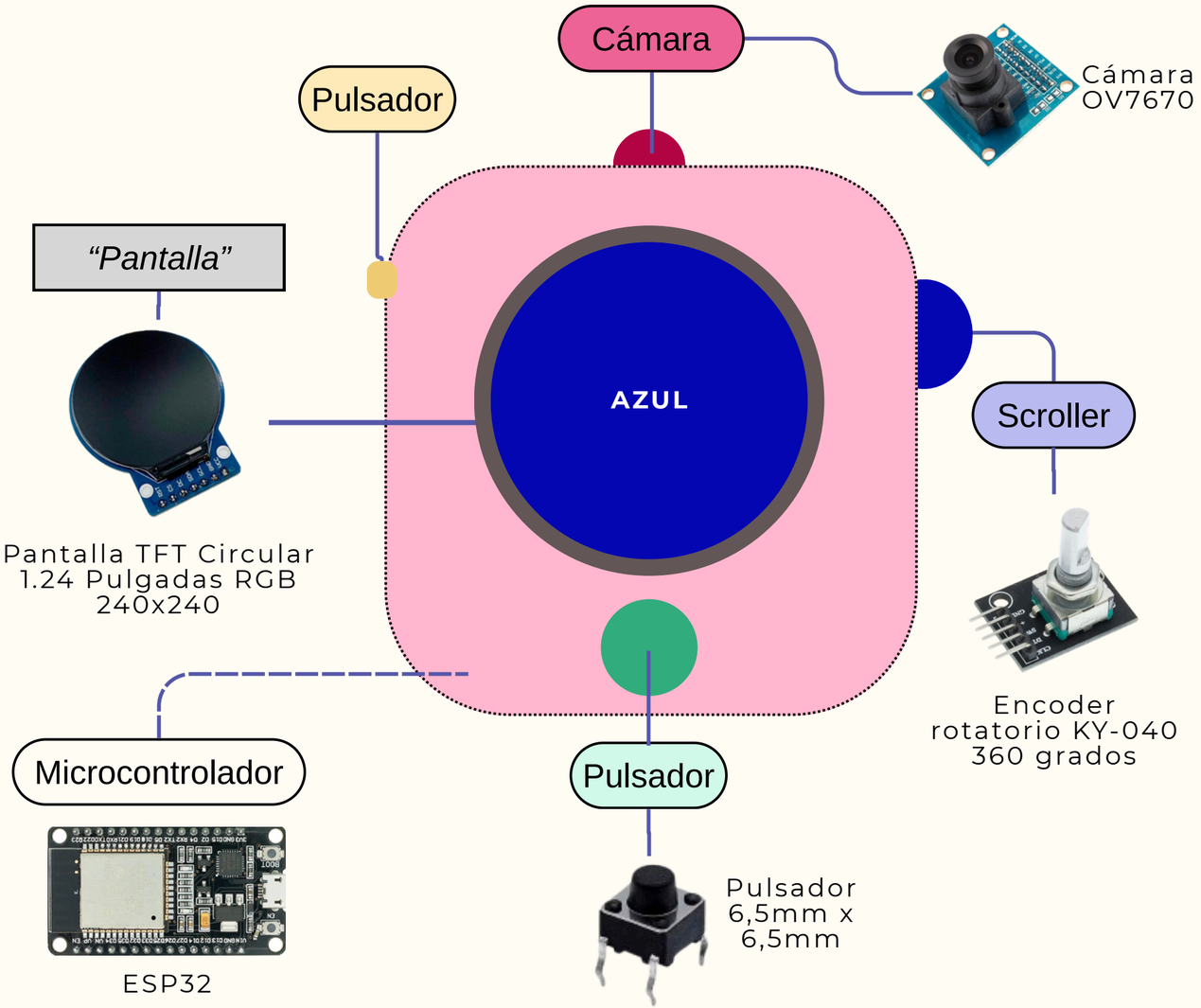
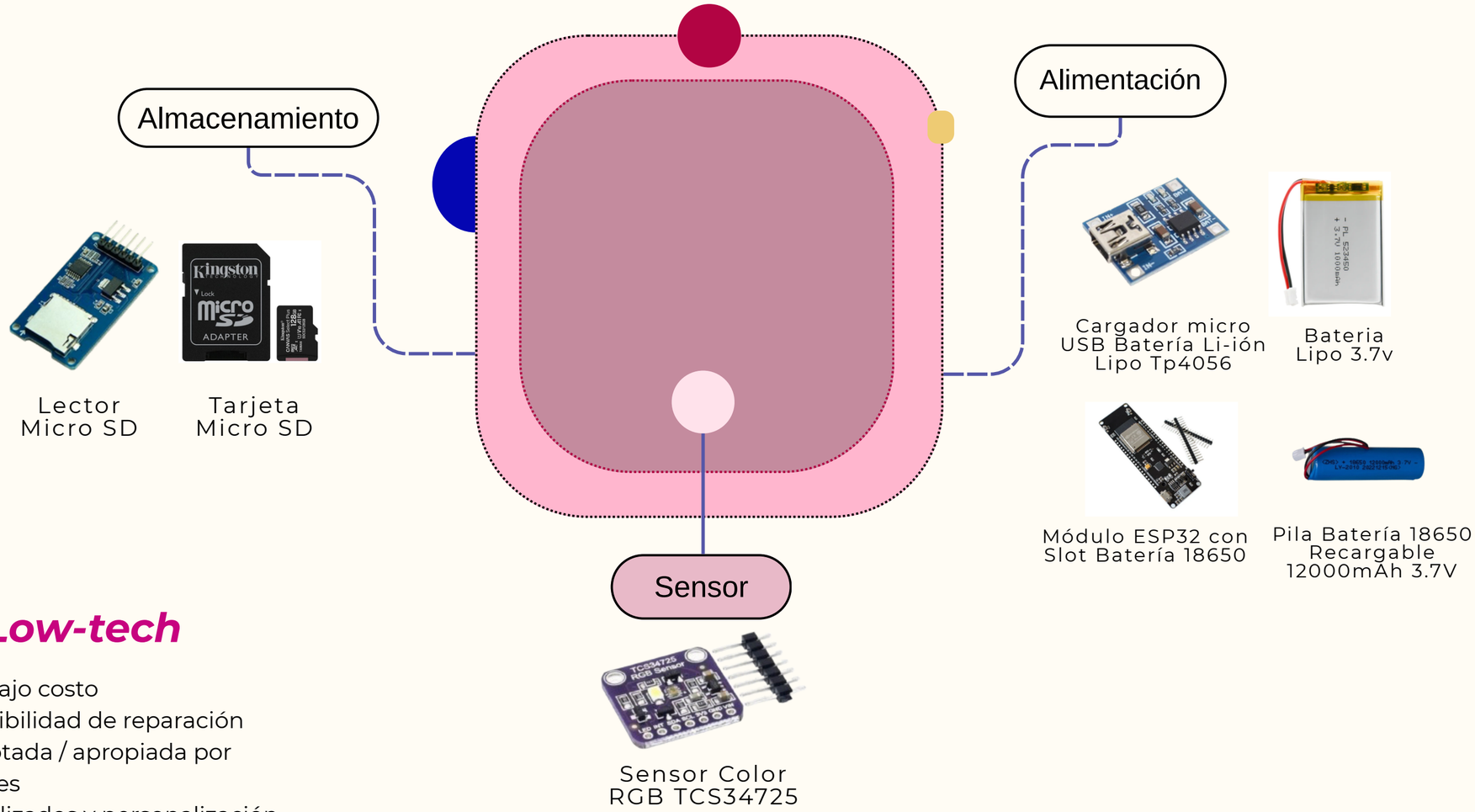


DIAGRAMA INICIAL Y COMPONENTES SISTEMA

Vista posterior



Aspectos Low-tech

- Componentes bajo costo
- Circuito con posibilidad de reparación
- Tecnología adaptada / apropiada por usuarios infantiles
- *Materiales reutilizados y personalización

FUNCIONAMIENTO DEL PROTOTIPO

Dispositivo tipo cámara digital que invita a niñas y niños a explorar y reconocer su entorno con una mirada distinta, la de buscar, reconocer y recolectar colores. Más que un dispositivo para registrar imágenes, se convierte en un compañero y mediador de juego que propone desafíos, acompaña la búsqueda, celebra los descubrimientos y promueve la exploración.

La búsqueda los obliga a moverse, observar con atención y fijarse en detalles que normalmente pasarían por alto.

01

La experiencia comienza cuando al iniciar la partida, el dispositivo anuncia un desafío, dando las respectivas pistas del color.

02

Desde ese momento comienza la exploración. Cada jugador con el dispositivo, sale a buscar en su entorno un objeto de ese color.

03

Al acercar el dispositivo el sensor analiza los colores y al presionar el botón disparador, la cámara captura y guarda la imagen con el color encontrado. Si coincide el color pedido con el encontrado, la pantalla se ilumina.

04

Al final de la partida. Las fotos quedan guardadas en la tarjeta de memoria, y el dispositivo permite recorrerlas como una galería.

Este momento invita a los niños a reunirse, mostrar lo que encontraron y conversar sobre sus hallazgos.

ESTRATEGIA DE INTERACCIÓN

Cazador de colores

Individual

Colores + aleatorio

Jugador → Selecciona modo de pistas aleatorias → Busca y explora hasta encontrar algo de ese color → Registra con la cámara → **Disp** Identifica el color → Lo agrega a la colección/archivo

Multijugador

Veo veo/Pistas/adivinanza

Jugador A Registra con la cámara un objeto de color → **Disp** Identifica el color → Guía las pistas:

- Es cálido o frío?
- Qué sensación da
- Hay alguna fruta o alimento de este color?
- Qué emocion
- Está cerca de...

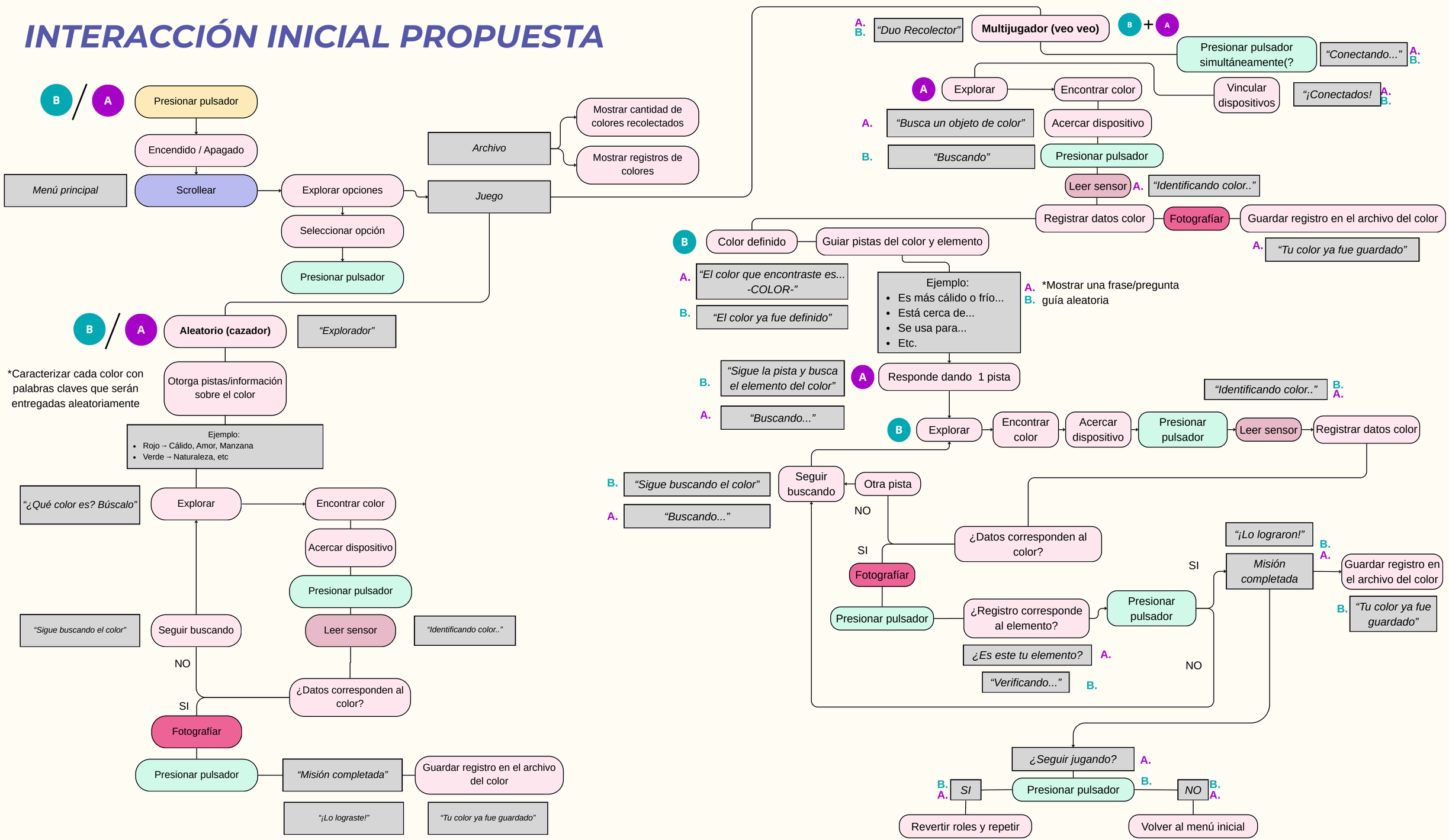
Jugador A

Contesta las preguntas/da pistas de lo que registró → Verifica el objeto/elemento

Jugador B

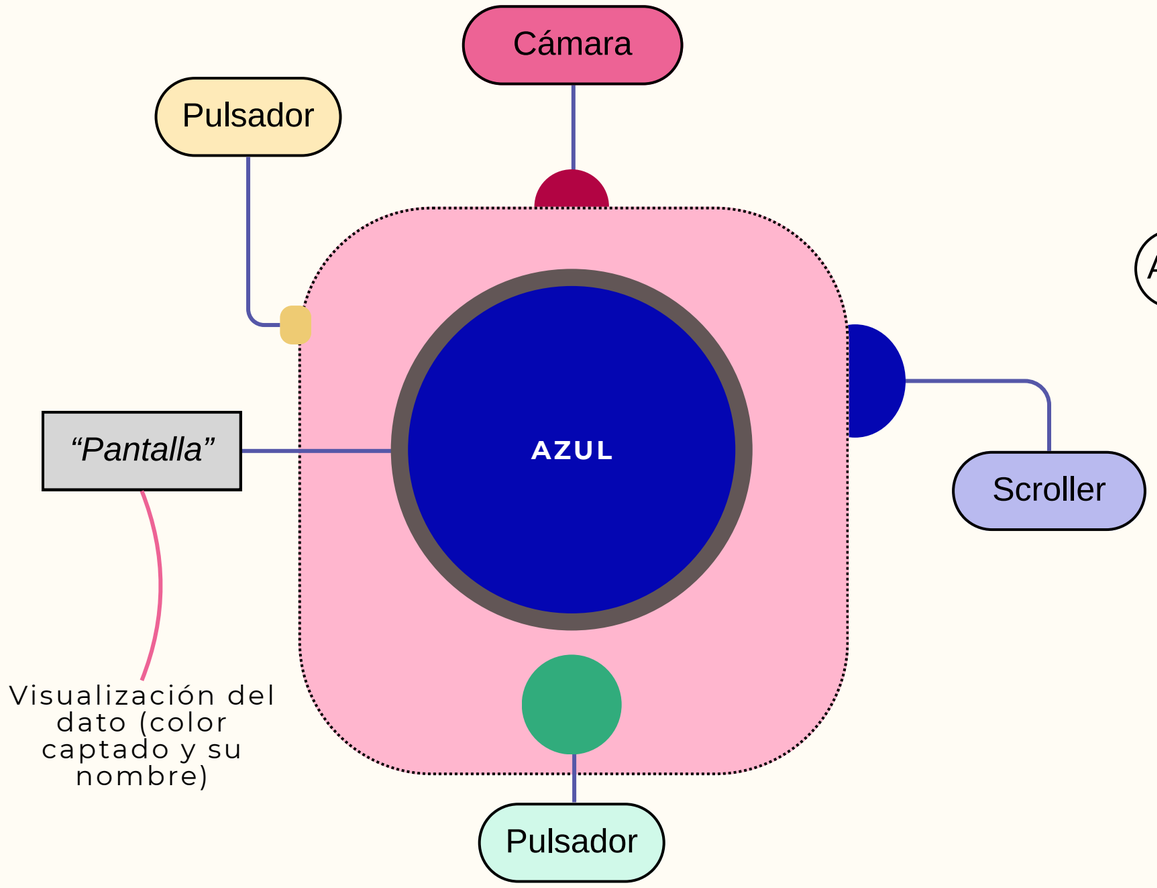
Busca y explora según las pistas → Registra el objeto y color → Identifica el color

INTERACCIÓN INICIAL PROPUESTA

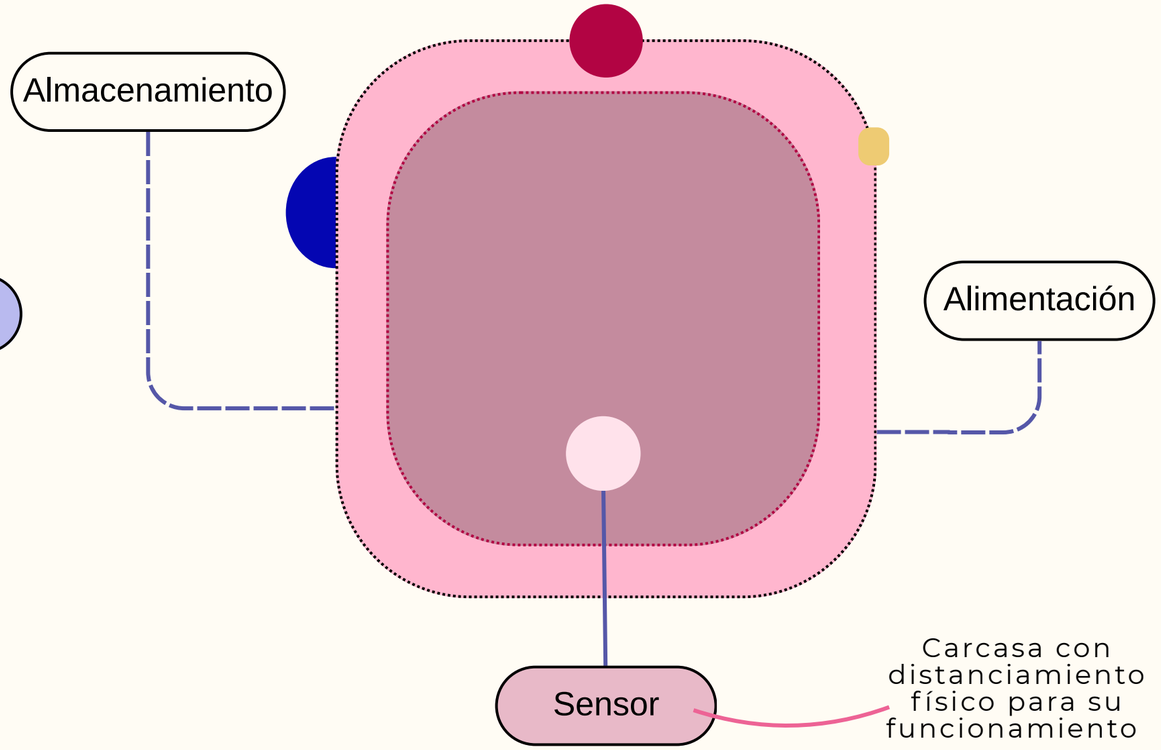


VISUALIZACIÓN DE LA PROPUESTA

Vista frontal



Vista posterior



VISUALIZACIÓN DE LA PROPUESTA

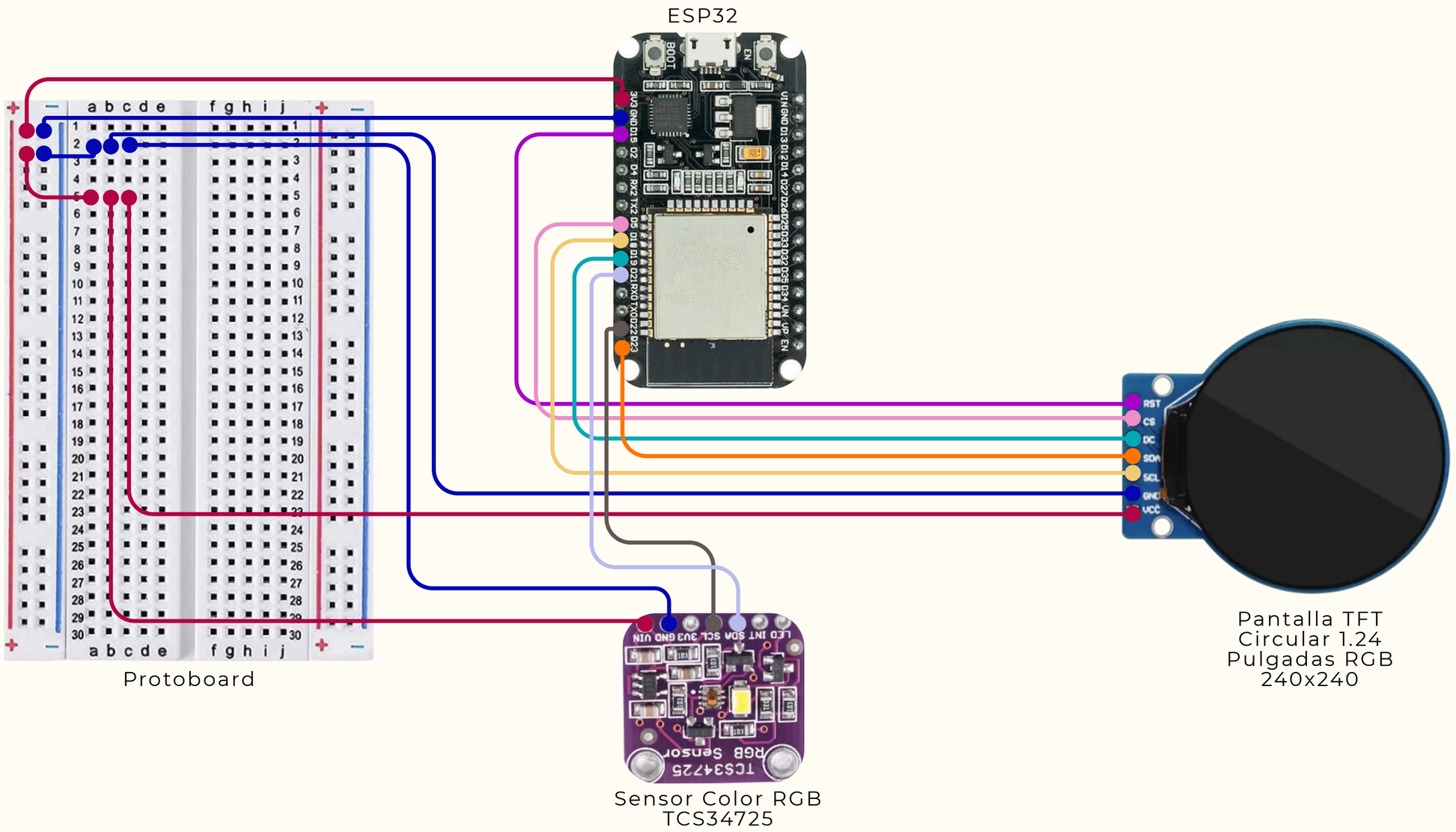
Render variación morfológica





*Vizcom

CIRCUITO ELECTRÓNICO



LÓGICA DE PROGRAMACIÓN (CÓDIGO)

1. **Prueba sensor RGB**
2. **Prueba Pantalla**
3. **Prueba traducción del RGB en la pantalla**
4. Prueba color y su nombre en pantalla

01

```
#include <Wire.h>
#include "Adafruit_TCS34725.h"

#define SENSOR_SDA 21
#define SENSOR_SCL 22

Adafruit_TCS34725 tcs =
  Adafruit_TCS34725(
    TCS34725_INTEGRATIONTIME_50MS,
    TCS34725_GAIN_4X
  );

void setup() {
  Serial.begin(115200);
  delay(1000);

  Serial.println();
  Serial.println("================================");
  Serial.println("PRUEBA 1 - TCS34725");
  Serial.println("================================");

  Wire.begin(SENSOR_SDA, SENSOR_SCL);

  Serial.println("Escaneando I2C...");

  byte encontrados = 0;

  for (byte address = 1; address < 127; address++) {
    Wire.beginTransmission(address);
    byte error = Wire.endTransmission();

    if (error == 0) {
      Serial.print("Dispositivo encontrado en 0x");
      if (address < 16) Serial.print("0");
      Serial.println(address, HEX);

      encontrados++;
    }
  }

  if (encontrados == 0) {
    Serial.println("NO SE ENCONTRO NINGUN DISPOSITIVO I2C.");
  }

  Serial.println();

  if (tcs.begin()) {
    Serial.println("OK: TCS34725 detectado.");
  } else {
    Serial.println("ERROR: TCS34725 NO detectado.");
    Serial.println("Revisa VCC, GND, SDA y SCL.");
  }
}

void loop() {

  uint16_t r, g, b, c;

  tcs.getRawData(&r, &g, &b, &c);

  Serial.print("R: ");
  Serial.print(r);

  Serial.print("  G: ");
  Serial.print(g);

  Serial.print("  B: ");
  Serial.print(b);

  Serial.print("  C: ");
  Serial.println(c);

  delay(500);
}
```

02

```
#include <Wire.h>
#include "Adafruit_TCS34725.h"
#include <Arduino_GFX_Library.h>

// ========================================================
// TCS34725
// ========================================================

#define SENSOR_SDA 21
#define SENSOR_SCL 22

Adafruit_TCS34725 tcs =
  Adafruit_TCS34725(
    TCS34725_INTEGRATIONTIME_50MS,
    TCS34725_GAIN_4X
  );

// ========================================================
// PANTALLA GC9A01 240x240
// ========================================================

#define TFT_DC   19
#define TFT_CS   5
#define TFT_SCK  18
#define TFT_MOSI 23
#define TFT_RST  15

Arduino_DataBus *bus = new Arduino_ESP32SPI(
  TFT_DC,
  TFT_CS,
  TFT_SCK,
  TFT_MOSI,
  GFX_NOT_DEFINED
);

Arduino_GFX *gfx = new Arduino_GC9A01(
  bus,
  TFT_RST,
  0,
  true
);

// ========================================================
// VARIABLES
// ========================================================

unsigned long ultimaLectura = 0;

const unsigned long INTERVALO = 10000; // 10 segundos

// ========================================================
// SETUP
// ========================================================

void setup() {

  Serial.begin(115200);
  delay(1000);

  Serial.println();
  Serial.println("================================");
  Serial.println("TCS34725 + PANTALLA GC9A01");
  Serial.println("================================");

  // ------------------------------------------------
  // INICIAR PANTALLA
  // ------------------------------------------------

  if (!gfx->begin(20000000)) {
    Serial.println("ERROR: No se pudo iniciar la pantalla.");
    while (true) {
      delay(1000);
    }
  }

  gfx->fillScreen(BLACK);

  Serial.println("Pantalla OK.");

  // ------------------------------------------------
  // INICIAR SENSOR
  // ------------------------------------------------

  Wire.begin(SENSOR_SDA, SENSOR_SCL);

  if (!tcs.begin()) {

    Serial.println("ERROR: TCS34725 NO detectado.");

    gfx->fillScreen(RED);

    while (true) {
      delay(1000);
    }
  }

  Serial.println("TCS34725 OK.");

  // ------------------------------------------------
  //
  // PRIMERA LECTURA
  // ------------------------------------------------

  actualizarColor();

  ultimaLectura = millis();
}

// ========================================================
// LOOP
// ========================================================

void loop() {

  if (millis() - ultimaLectura >= INTERVALO) {

    ultimaLectura = millis();

    actualizarColor();
  }

  delay(20);
}

// ========================================================
// LEER SENSOR Y MOSTRAR COLOR
// ========================================================

void actualizarColor() {

  uint16_t r;
  uint16_t g;
  uint16_t b;
  uint16_t c;

  tcs.getRawData(&r, &g, &b, &c);

  Serial.println();
  Serial.println("------------------------------");

  Serial.print("R: ");
  Serial.println(r);

  Serial.print("G: ");
  Serial.println(g);

  Serial.print("B: ");
  Serial.println(b);

  Serial.print("C: ");
  Serial.println(c);

  // ------------------------------------------------
  // EVITAR DIVISION POR CERO
  // ------------------------------------------------

  if (c == 0) {
    Serial.println("Sin luz detectada.");
    gfx->fillScreen(BLACK);
    return;
  }

  // ------------------------------------------------
  // NORMALIZAR RGB
  // ------------------------------------------------

  int rojo = (r * 255L) / c;
  int verde = (g * 255L) / c;
  int azul = (b * 255L) / c;

  // Limitar valores
  rojo = constrain(rojo, 0, 255);
  verde = constrain(verde, 0, 255);
  azul = constrain(azul, 0, 255);

  Serial.print("RGB pantalla: ");
  Serial.print(rojo);
  Serial.print(", ");
  Serial.print(verde);
  Serial.print(", ");
  Serial.println(azul);

  // ------------------------------------------------
  // MOSTRAR COLOR EN PANTALLA
  // ------------------------------------------------

  uint16_t colorPantalla = gfx->color565(
    rojo,
    verde,
    azul
  );

  gfx->fillScreen(colorPantalla);

  Serial.println("Color mostrado en pantalla.");
}
```

03

```
#include <Wire.h>
#include "Adafruit_TCS34725.h"
#include <Arduino_GFX_Library.h>

//
// ================================================
// TCS34725
//
// ================================================

#define SENSOR_SDA 21
#define SENSOR_SCL 22

Adafruit_TCS34725 tcs =
  Adafruit_TCS34725(
    TCS34725_INTEGRATIONTIME_50MS,
    TCS34725_GAIN_4X
  );

//
// ================================================
// PANTALLA GC9A01 240x240
//
// ================================================

#define TFT_DC   19
#define TFT_CS   5
#define TFT_SCK  18
#define TFT_MOSI 23
#define TFT_RST  15

Arduino_DataBus *bus = new Arduino_ESP32SPI(
  TFT_DC,
  TFT_CS,
  TFT_SCK,
  TFT_MOSI,
  GFX_NOT_DEFINED
);

Arduino_GFX *gfx = new Arduino_GC9A01(
  bus,
  TFT_RST,
  0,
  true
);

//
// ================================================
// VARIABLES
//
// ================================================

unsigned long ultimaLectura = 0;

const unsigned long INTERVALO = 10000; // 10 segundos

//
// ================================================
// SETUP
//
// ================================================

void setup() {

  Serial.begin(115200);
  delay(1000);

  Serial.println();
  Serial.println("================================");
  Serial.println("TCS34725 + PANTALLA GC9A01");
  Serial.println("================================");

  // ------------------------------------------------
  // INICIAR PANTALLA
  // ------------------------------------------------

  if (!gfx->begin(20000000)) {

    Serial.println("ERROR: No se pudo iniciar la pantalla.");

    while (true) {
      delay(1000);
    }
  }

  gfx->fillScreen(RGB565_BLACK);

  Serial.println("Pantalla OK.");

  // ------------------------------------------------
  // INICIAR SENSOR
  // ------------------------------------------------

  Wire.begin(SENSOR_SDA, SENSOR_SCL);

  if (!tcs.begin()) {

    Serial.println("ERROR: TCS34725 NO detectado.");

    gfx->fillScreen(RGB565_RED);

    while (true) {
      delay(1000);
    }
  }

  Serial.println("TCS34725 OK.");

  // ------------------------------------------------
  // PRIMERA LECTURA
  // ------------------------------------------------

  actualizarColor();

  ultimaLectura = millis();
}

//
// ================================================
// LOOP
//
// ================================================

void loop() {

  if (millis() - ultimaLectura >= INTERVALO) {

    ultimaLectura = millis();

    actualizarColor();
  }

  delay(20);
}

//
// ================================================
// LEER SENSOR Y MOSTRAR COLOR
//
// ================================================

void actualizarColor() {

  uint16_t r;
  uint16_t g;
  uint16_t b;
  uint16_t c;

  tcs.getRawData(&r, &g, &b, &c);

  Serial.println();
  Serial.println("------------------------------");

  Serial.print("R: ");
  Serial.println(r);

  Serial.print("G: ");
  Serial.println(g);

  Serial.print("B: ");
  Serial.println(b);

  Serial.print("C: ");
  Serial.println(c);

  // ------------------------------------------------
  // EVITAR DIVISION POR CERO
  // ------------------------------------------------

  if (c == 0) {

    Serial.println("Sin luz detectada.");

    gfx->fillScreen(RGB565_BLACK);

    return;
  }

  // ------------------------------------------------
  // NORMALIZAR RGB
  // ------------------------------------------------

  int rojo = (r * 255L) / c;
  int verde = (g * 255L) / c;
  int azul = (b * 255L) / c;

  // Limitar valores
  rojo = constrain(rojo, 0, 255);
  verde = constrain(verde, 0, 255);
  azul = constrain(azul, 0, 255);

  Serial.print("RGB pantalla: ");
  Serial.print(rojo);
  Serial.print(", ");
  Serial.print(verde);
  Serial.print(", ");
  Serial.println(azul);

  // ------------------------------------------------
  // MOSTRAR COLOR EN PANTALLA
  // ------------------------------------------------

  uint16_t colorPantalla = gfx->color565(
    rojo,
    verde,
    azul
  );

  gfx->fillScreen(colorPantalla);

  Serial.println("Color mostrado en pantalla.");
}
```

LÓGICA DE PROGRAMACIÓN (CÓDIGO)

1. Prueba sensor RGB
2. Prueba Pantalla
3. Prueba traducción del RGB en la pantalla
4. **Prueba color y su nombre en pantalla**


```
#include <Wire.h>
#include "Adafruit_TCS34725.h"
#include <Arduino_GFX_Library.h>

// ========================================================
// TCS34725
// ========================================================

#define SENSOR_SDA 21
#define SENSOR_SCL 22

Adafruit_TCS34725 tcs =
  Adafruit_TCS34725(
    TCS34725_INTEGRATIONTIME_50MS,
    TCS34725_GAIN_4X
  );

// ========================================================
// PANTALLA GC9A01 240x240
// ========================================================

#define TFT_DC    19
#define TFT_CS    5
#define TFT_SCK   18
#define TFT_MOSI  23
#define TFT_RST   15

Arduino_DataBus *bus = new Arduino_ESP32SPI(
  TFT_DC,
  TFT_CS,
  TFT_SCK,
  TFT_MOSI,
  GFX_NOT_DEFINED
);

Arduino_GFX *gfx = new Arduino_GC9A01(
  bus,
  TFT_RST,
  0,
  true
);

// ========================================================
// TIEMPO
// ========================================================

unsigned long ultimaLectura = 0;

// CAMBIO CADA 5 SEGUNDOS
const unsigned long INTERVALO = 5000;


// ========================================================
// SETUP
// ========================================================

void setup() {

  Serial.begin(115200);
  delay(1000);

  Serial.println();
  Serial.println("================================");
  Serial.println("CAZADOR DE COLORES");
  Serial.println("TCS34725 + GC9A01");
  Serial.println("================================");

  // ----------------------------------------------
  // PANTALLA
  // ----------------------------------------------

  if (!gfx->begin(20000000)) {

    Serial.println("ERROR: No se pudo iniciar la pantalla.");

    while (true) {
      delay(1000);
    }
  }

  gfx->fillScreen(RGB565_BLACK);

  Serial.println("Pantalla OK.");

  // ----------------------------------------------
  // SENSOR
  // ----------------------------------------------

  Wire.begin(SENSOR_SDA, SENSOR_SCL);

  if (!tcs.begin()) {

    Serial.println("ERROR: TCS34725 NO detectado.");

    gfx->fillScreen(RGB565_RED);

    while (true) {
      delay(1000);
    }
  }

  Serial.println("TCS34725 OK.");

  // ----------------------------------------------
  // PRIMERA LECTURA
  // ----------------------------------------------

  actualizarColor();

  ultimaLectura = millis();
}


// ========================================================
// LOOP
// ========================================================

void loop() {

  if (millis() - ultimaLectura >= INTERVALO) {

    ultimaLectura = millis();

    actualizarColor();
  }

  delay(20);
}


// ========================================================
// DETECTAR NOMBRE DEL COLOR
// ========================================================

const char* nombreColor(
  int rojo,
  int verde,
  int azul
) {

  // ----------------------------------------------
  // NEGRO
  // ----------------------------------------------

  if (rojo < 30 &&
      verde < 30 &&
      azul < 30) {

    return "NEGRO";
  }


  // ----------------------------------------------
  // BLANCO
  // ----------------------------------------------

  if (rojo > 200 &&
      verde > 200 &&
      azul > 200) {

    return "BLANCO";
  }


  // ----------------------------------------------
  // AMARILLO
  // ----------------------------------------------

  if (rojo > 150 &&
      verde > 130 &&
      azul < 100) {

    return "AMARILLO";
  }


  // ----------------------------------------------
  // NARANJA
  // ----------------------------------------------

  if (rojo > 170 &&
      verde > 70 &&
      verde < 160 &&
      azul < 80) {

    return "NARANJA";
  }


  // ----------------------------------------------
  // ROJO
  // ----------------------------------------------

  if (rojo > verde * 1.4 &&
      rojo > azul * 1.4) {

    return "ROJO";
  }


}
  // ----------------------------------------------
  // VERDE
  // ----------------------------------------------

  if (verde > rojo * 1.3 &&
      verde > azul * 1.2) {

    return "VERDE";
  }


  // ----------------------------------------------
  // AZUL
  // ----------------------------------------------

  if (azul > rojo * 1.3 &&
      azul > verde * 1.15) {

    return "AZUL";
  }


  // ----------------------------------------------
  // CIAN
  // ----------------------------------------------

  if (verde > rojo * 1.15 &&
      azul > rojo * 1.15) {

    return "CIAN";
  }


  // ----------------------------------------------
  // MORADO
  // ----------------------------------------------

  if (rojo > verde * 1.15 &&
      azul > verde * 1.15) {

    return "MORADO";
  }


  // ----------------------------------------------
  // COLOR NO CLASIFICADO
  // ----------------------------------------------

  return "COLOR";
}


// ========================================================
// LEER SENSOR Y MOSTRAR
// ========================================================

void actualizarColor() {

  uint16_t r;
  uint16_t g;
  uint16_t b;
  uint16_t c;

  // Leer sensor
  tcs.getRawData(&r, &g, &b, &c);


  // ----------------------------------------------
  // SERIAL
  // ----------------------------------------------

  Serial.println();
  Serial.println("----------------------------");

  Serial.print("R: ");
  Serial.println(r);

  Serial.print("G: ");
  Serial.println(g);

  Serial.print("B: ");
  Serial.println(b);

  Serial.print("C: ");
  Serial.println(c);


  // ----------------------------------------------
  // SIN LUZ
  // ----------------------------------------------

  if (c == 0) {

    Serial.println("Sin luz detectada.");

    gfx->fillScreen(RGB565_BLACK);

    gfx->setTextSize(2);
    gfx->setTextColor(RGB565_WHITE);

    gfx->setCursor(70, 110);

    gfx->println("SIN LUZ");

    return;
  }


  // ----------------------------------------------
  // NORMALIZAR RGB
  // ----------------------------------------------

  int rojo  = (r * 255L) / c;
  int verde = (g * 255L) / c;
  int azul  = (b * 255L) / c;


  // Limitar valores
  rojo  = constrain(rojo, 0, 255);
  verde = constrain(verde, 0, 255);
  azul  = constrain(azul, 0, 255);


  // ----------------------------------------------
  // OBTENER NOMBRE DEL COLOR
  // ----------------------------------------------

  const char* nombre = nombreColor(
    rojo,
    verde,
    azul
  );


  // ----------------------------------------------
  // SERIAL RGB
  // ----------------------------------------------

  Serial.print("RGB: ");

  Serial.print(rojo);
  Serial.print(", ");

  Serial.print(verde);
  Serial.print(", ");

  Serial.println(azul);


  Serial.print("COLOR DETECTADO: ");

  Serial.println(nombre);


  // ----------------------------------------------
  // COLOR DE FONDO
  // ----------------------------------------------

  uint16_t colorPantalla = gfx->color565(
    rojo,
    verde,
    azul
  );

  gfx->fillScreen(colorPantalla);


  // ----------------------------------------------
  // CALCULAR LUMINOSIDAD
  // ----------------------------------------------

  int luminosidad =
    (rojo * 299 +
     verde * 587 +
     azul * 114) / 1000;


  // ----------------------------------------------
  // COLOR DEL TEXTO
  // ----------------------------------------------

  uint16_t colorTexto;

  if (luminosidad > 150) {

    // Fondo claro
    colorTexto = RGB565_BLACK;

  } else {

    // Fondo oscuro
    colorTexto = RGB565_WHITE;
  }


  // ----------------------------------------------
  // MOSTRAR NOMBRE
  // ----------------------------------------------

  gfx->setTextSize(3);

  gfx->setTextColor(colorTexto);


  // Ancho aproximado del texto
  int anchoTexto = strlen(nombre) * 18;

  int x = (240 - anchoTexto) / 2;

  if (x < 5) {
    x = 5;
  }


  // Posición vertical
  gfx->setCursor(x, 105);

  gfx->println(nombre);

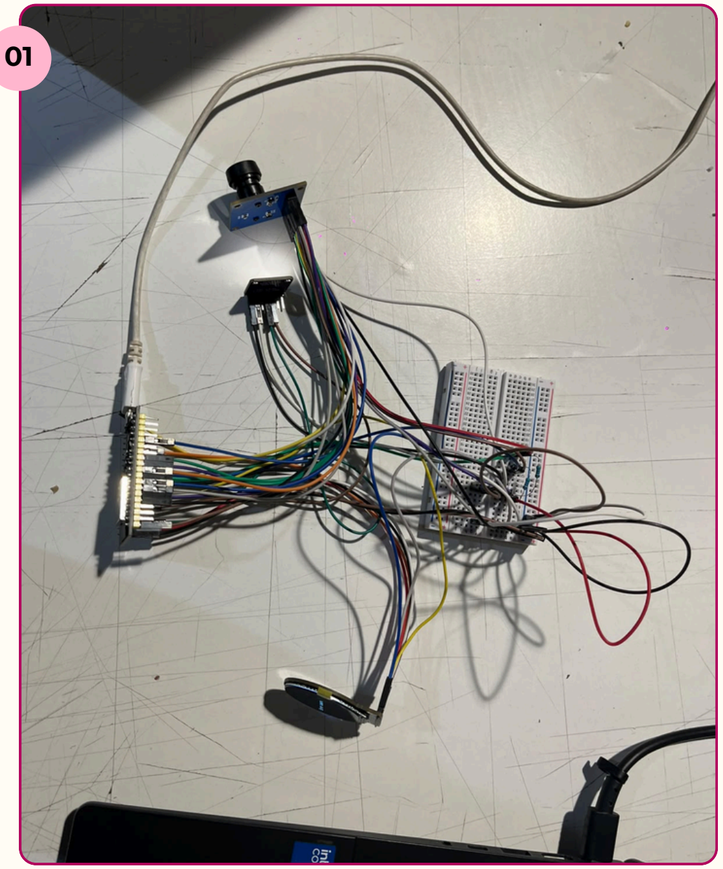

  // ----------------------------------------------
  // SERIAL
  // ----------------------------------------------

  Serial.println("Color mostrado en pantalla.");

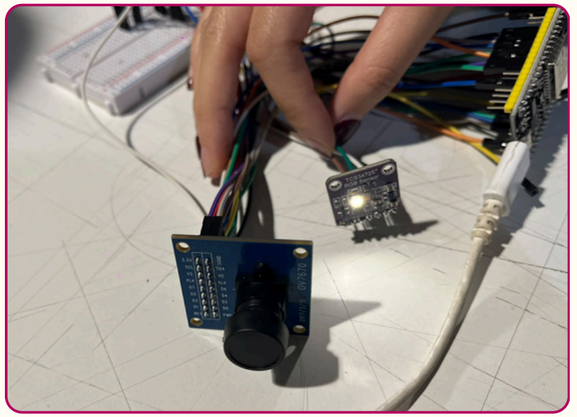
  Serial.println("Proxima lectura en 5 segundos.");
}
```

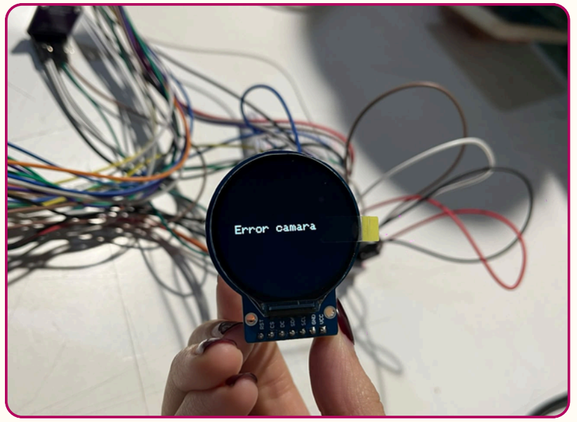
PRUEBAS INICIALES

01

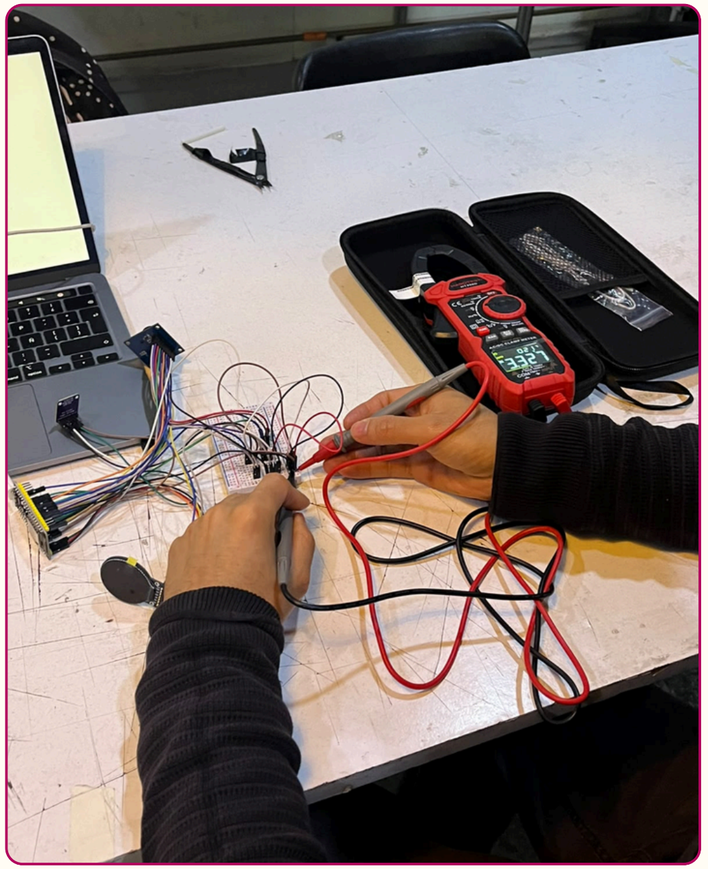


01. Se conectaron todos los componentes a utilizar: Cámara, Pantalla y Sensor a la Esp32 utilizando una protoboard





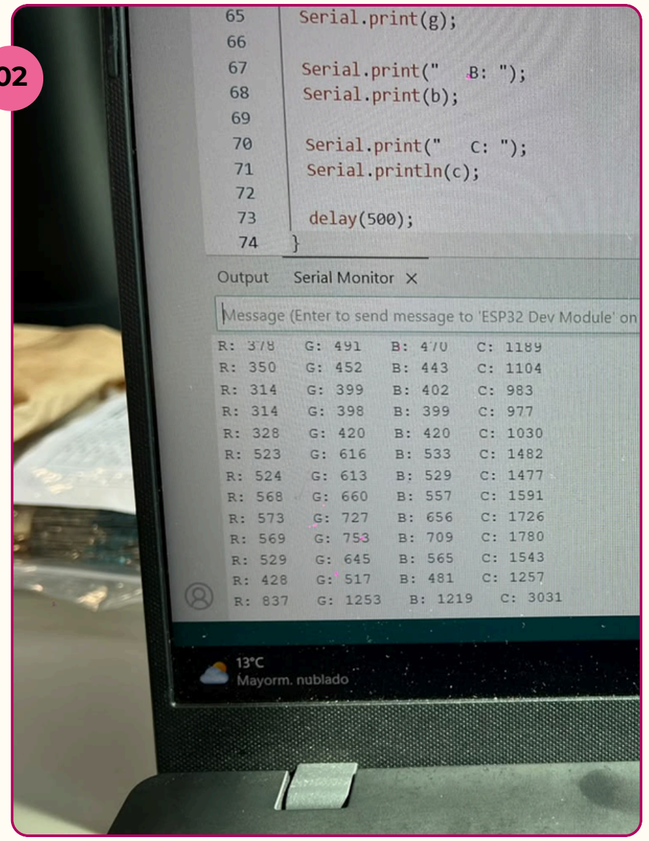
02. ERROR: No se detectó la Cámara ni el sensor



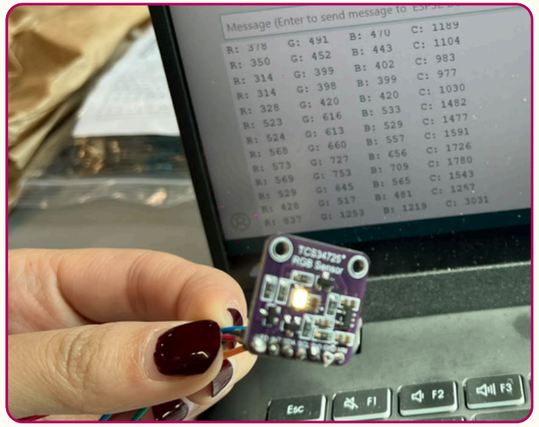
03. Se corroboró el estado de los cables y la alimentación, al verificarse su correcto funcionamiento se evaluó el funcionamiento de cada componente individualmente

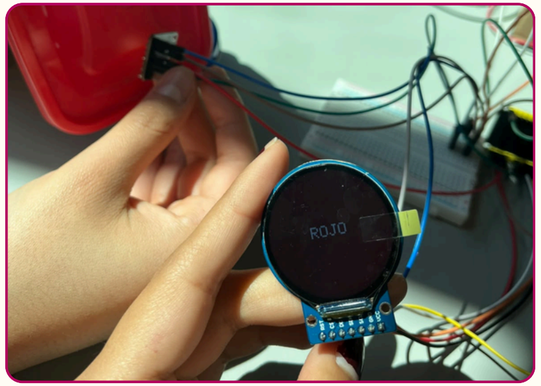
PRUEBAS INICIALES

02

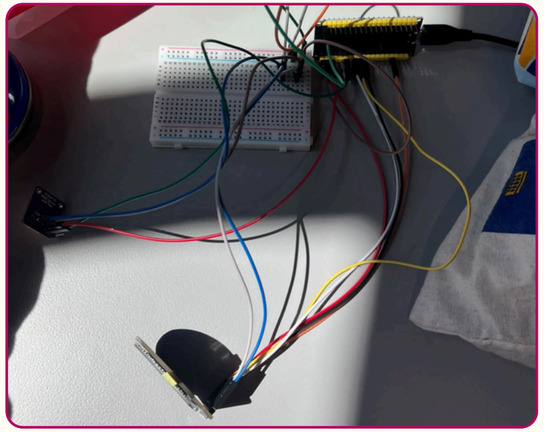


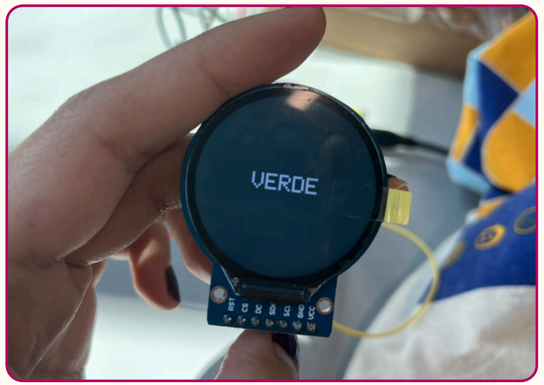
01. Al verificar nuevamente se obtuvo que el error estaba en el reconocimiento del Sensor. Se revisó su funcionamiento y se corrigió la soldadura de los pines





02. Se logró el correcto funcionamiento del Sensor y se desarrollaron los correspondientes códigos de prueba en conjunto con Claude para la detección de colores primarios.



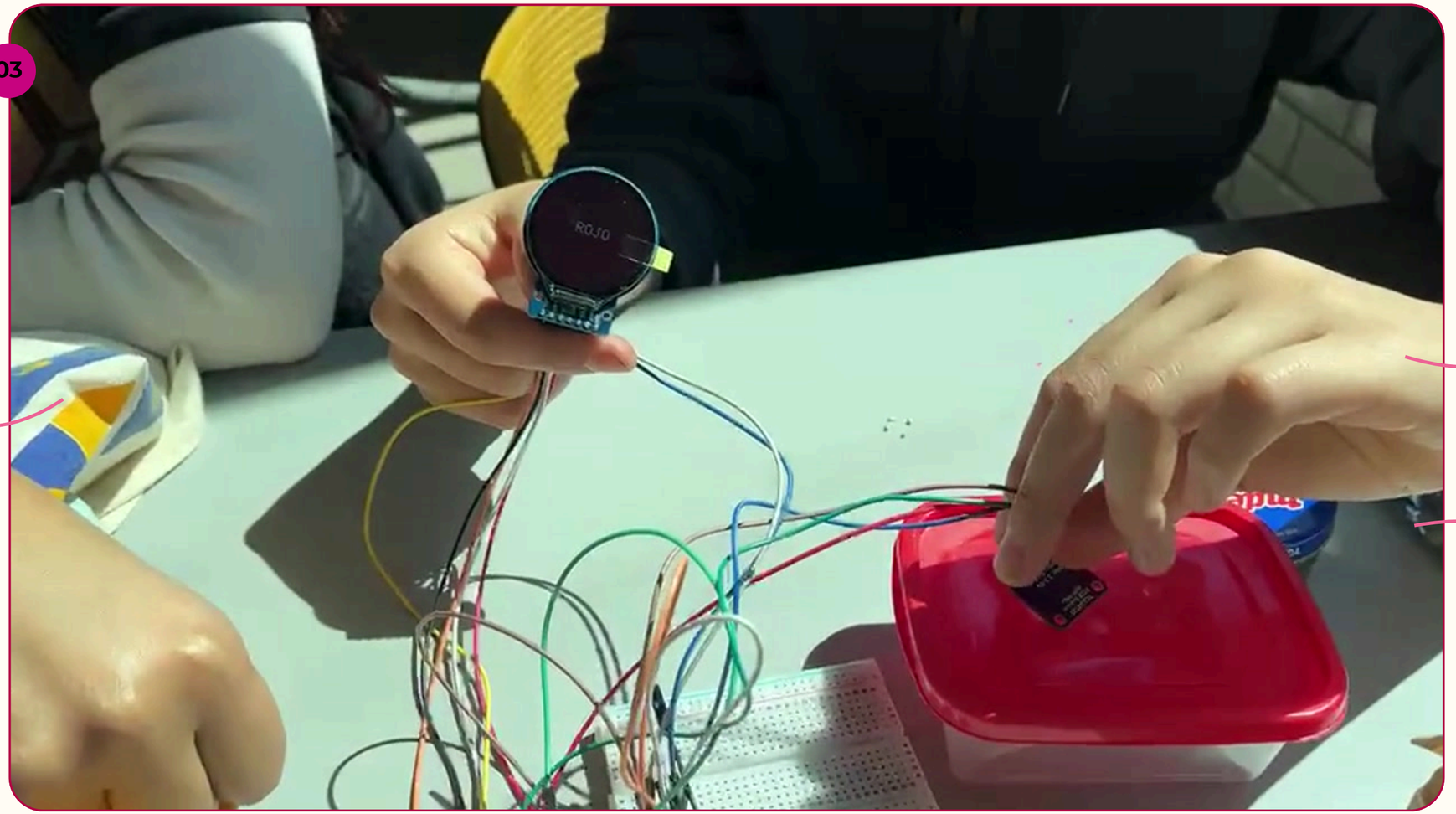


03. Se realizaron modificaciones al código en cuanto a detección de colores secundarios, regulación del sensor y visibilización de datos

PRUEBAS INICIALES

03

Prueba final funcionamiento



Se detectan colores a distancias cortas

Elemento a considerar en la carcasa física

Parametrización de detección de colores primario y secundarios

Se debe considerar la influencia de la iluminación ambiente al captar colores

Gracias por su atención!